

Quality Lab Manual



Operating System Lab

2ICPC204

Experiment No. 4

Program based on various I/O System calls of UNIX operating System.

Course Instructor -

PROF.MADHURI J SIDDHARAPU
ASSISTANT PROFESSOR

Experiment No. 4

Title of Experiment: Program based on various I/O System calls of UNIX operating System.

Aim of Experiment: Write a C program to implement I/O System calls of UNIX operating System.

System Requirements –

Operating System: UNIX or Linux-based system (e.g., Ubuntu, CentOS, macOS)

Editor: Nano editor (available by default on many UNIX/Linux systems)

Compiler: GCC (GNU Compiler Collection) version 4.8 or later

Hardware: Minimum 512 MB RAM and a basic CPU (e.g., Intel Core i3 or equivalent)

Disk Space: Minimal, around 10 MB for source code and temporary files

Description -

Procedure:

1. Install UNIX or Linux-based operating system installed, such as Ubuntu.
2. Use Nano editor and install GCC compiler. Check version by running `gcc --version` in the terminal.
3. Open the Nano Editor by typing `nano` and the file name in terminal.
4. Write the C program and save the file by pressing **CTRL + o**, then press **Enter**. Exit the Nano editor by pressing **CTRL + x**.
5. Compile the program in the terminal using the GCC compiler by typing `gcc filename`.
6. Execute the compiled program by typing `./a.out filename`

Program :

1) Write System Call

Syntax:

```
#include<unistd.h>
ssize_t write(int fd, const void *buf, size_t count);
```

The first parameter (fd) is the file descriptor where you want to write. The data that is to be written is specified in the second parameter. Finally, the third parameter is the total bytes that are to be written.

Program : write some data on the standard output device (by default – monitor)

//Name the program file as “write.c” and save Ctrl + X in nano editor

```
#include<unistd.h>
int main()
{
write(1,"hello\n",6);
}
```

//1 is the file descriptor, "hello\n" is the data, 6 is the count of characters in data //

Output:

2) Read System Call

The use of *read()* system call is to read from a file descriptor. The working is same as *write()*, the only difference is *read()* will read the data from file pointed to by file descriptor.

Syntax:

```
#include<unistd.h>
ssize_t read(int fd, const void *buf, size_t count);
```

The first parameter is the file descriptor. The second parameter is the buffer where the read data

will be saved. Lastly, the third parameter is the number of bytes that you want to read.

Program:

//Name the program file as “read.c”

```
//read.c

#include<unistd.h>
int main()
{
    char buff[20];
    read(0,buff,10);    //read 10 bytes from standard input device(keyboard), store in buffer
                        (buff) //
    write(1,buff,10);    //print 10 bytes from the buffer on the screen
}
```

Output :

3) Open System Call

To Write a program using open() system call to read the first 30 characters of an existing file “test.txt” and print them on screen.

```
//open.c
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
int main()
{
    int n,fd;
    char buff[50];
    fd=open("test.txt",O_RDONLY);

                                //opens test.txt in read mode and the file descriptor is
                                saved in integer fd. //

    n=read(fd,buff,30);        //read 30 characters from the file pointed to by file descriptor fd
                                and save them in buffer (buff)//
    write(1,buff,n);           //write on the screen from the buffer
}
```

working

First, create a file “test.txt” and write some content into it(more than 30 characters). The open() system call opens the file test.txt in read-only mode and returns the file descriptor. This file

descriptor is saved in variable 'fd'. Next, use read() to read 30 characters from the file into the buffer. Finally, the buffer values are printed on the screen. The file descriptor used here is '0' which is the file descriptor for standard output device.

Output

4) fork System Call

Syntax :

```
#include<unistd.h>
pid_t fork(void);
```

fork() returns -1 on failure; On success it returns '0' in the child process and process-id of the child in the parent process.

Program for fork() system call

```
#include<stdio.h>
#include<unistd.h>
#include<sys/types.h>
int main()
{
    pid_t p;
    printf("before fork\n");

    p=fork();
    if(p==0)
    {
        printf("I am child having id %d\n",getpid());
        printf("My parent's id is %d\n",getppid());

    }
    else
    {
        printf("My child's id is %d\n",p);
        printf("I am parent having id %d\n",getpid());
    }
    printf("Common\n");
}
```

Output

wait System Call

Syntax :

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t wait(int *wstatus);
```

wait() system call takes only one parameter which stores the status information of the process. Pass NULL as the value if you do not want to know the exit status of the child process and are simply concerned with making the parent wait for the child. On success, wait returns the PID of the terminated child process while on failure it returns -1.

Important: wait() can be used to make the parent wait for the child to terminate(finish) but not the other way around.

Program:

```
#include<unistd.h>
#include<sys/types.h>
#include<stdio.h>
#include<sys/wait.h>
int main()
{
    pid_t p;
    printf("before fork\n");
    p=fork();
    if(p==0)//child
    {
        printf("I am child having id %d\n",getpid());
        printf("My parent's id is %d\n",getppid());
    }
    else //parent
    {
        wait(NULL);
        printf("My child's id is %d\n",p);
        printf("I am parent having id %d\n",getpid());
    }
    printf("Common\n");
}
```

Output

Conclusion

Thus I/O System calls of UNIX operating System are successfully implemented.

